Paper Title

Given Name Surname
dept. name of organization
(of Affiliation)
name of organization
(of Affiliation)
City, Country
email address or ORCID

***Abstract*— ROS 2 (Robot Operating System 2) is a middleware framework for developing modular and distributed robotic applications. It enables independent software components to communicate through standardised interfaces. This report examines the ROS 2 computation graph, nodes, topics, services, actions, parameters, custom interfaces, workspace organisation, the colcon build system, and Quality of Service (QoS). Particular attention is given to the differences between communication mechanisms, their appropriate applications, and the role of middleware in ensuring compatible data exchange. Practical command-line examples and conceptual architecture diagrams are included to demonstrate how ROS 2 components interact. The report provides a conceptual foundation for understanding and developing ROS 2 applications using the Humble distribution..**

Keywords*— *ROS 2 Humble, computation graph, nodes, topics, services, actions, parameters, custom interfaces, colcon, Quality of Service.

1. INTRODUCTION

Modern robotic systems comprise several interconnected subsystems responsible for sensing, localisation, perception, planning, control, and human–robot interaction. Implementing all these functions within a single program can make the software difficult to maintain, test, and extend.

ROS 2 addresses this challenge by providing a framework for developing modular robotic applications. Individual computational components are implemented as nodes that communicate through standardised interfaces. This architecture allows developers to modify, replace, or test individual components without necessarily redesigning the entire application.

The ROS 2 computation graph represents the nodes in a running system and their communication relationships. Understanding this graph is fundamental to developing reliable robotic applications.

The objectives of this report are to:

1. Explain the ROS 2 computation graph and middleware architecture.
2. Describe nodes, topics, services, actions, and parameters.
3. Compare topics and services and explain when each should be used.
4. Explain custom message, service, and action interfaces.
5. Describe ROS 2 workspace organisation and the colcon build system.
6. Explain Quality of Service policies and their influence on communication.
7. Demonstrate the concepts through practical command-line examples.

2. II. ROS 2 COMPUTATION GRAPH AND MIDDLEWARE

A. Computation Graph

The ROS 2 computation graph is a runtime representation of the nodes in a robotic system and the communication relationships between them. Nodes perform computational tasks, while communication interfaces allow information to flow between nodes.

For example, an autonomous vehicle may contain a sensor node, a perception node, a planning node, and a controller node. The sensor node publishes measurements, the perception node processes them, the planning node determines a trajectory, and the controller generates commands.

These components can operate independently while collaborating through ROS 2 communication mechanisms.

The computation graph is dynamic. Nodes may be added or removed as processes start or stop, and the graph reflects the communication interfaces exposed by the running system.

Conceptual ROS 2 Computation Graph

Sensor Node

Publishes sensor measurements

Publish

Topic: /sensor_data

Typed asynchronous messages

Subscribe

Processing Node

Processes incoming measurements

Service

Request and response

Action

Goal, feedback and result

Fig. 1. Conceptual representation of ROS 2 nodes and communication interfaces.

Middleware Architecture

ROS 2 commonly uses Data Distribution Service (DDS) implementations through the ROS Middleware Interface

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

(RMW). DDS provides communication and discovery mechanisms that enable compatible endpoints to find one another and exchange data.

The middleware abstracts many transport and discovery details from the application developer. Consequently, developers can focus on the logic of individual nodes and the interfaces connecting them.

Communication compatibility is influenced by factors such as topic names, message types, namespaces, ROS domain configuration, and QoS settings.

Useful commands for inspecting the computation graph include:

ros2 node list

ros2 topic list

ros2 service list

ros2 action list

The rqt_graph graphical tool displays node and topic connections. It is useful for identifying communication relationships, but the presence of a connection does not independently prove that the application is producing correct results.

3. NODES

A node is a logical unit of computation in a ROS 2 application. Each node is generally designed to perform a specific function, such as reading a sensor, calculating wheel speed, estimating position, planning a trajectory, or controlling an actuator.

A node may expose several interfaces:

- Publishers for sending messages.
- Subscribers for receiving messages.
- Service servers for processing requests.
- Service clients for sending requests.
- Action servers and clients for goal-oriented operations.
- Parameters for configuring its behaviour.

For example, a wheel-speed calculation system could contain a publisher node that transmits angular velocity and wheel radius, and a subscriber node that calculates longitudinal speed.

Separating these functions into nodes improves modularity, facilitates testing, and allows components to be reused in different applications.

Inspecting Nodes

The following commands demonstrate node inspection:

ros2 node list

This lists the active nodes visible to the ROS 2 environment.

ros2 node info /turtlesim

This displays the interfaces associated with a node named /turtlesim, provided that it is running under that name.

Node names and namespaces help distinguish components within larger systems. A well-designed application should use meaningful node names that reflect their functions.

4. TOPICS

A topic is a named communication channel through which messages of a defined type are exchanged. ROS 2 uses a publish–subscribe communication model for topics.

A publisher sends messages without waiting for a particular subscriber to return a response. Subscribers receive messages when compatible publishers and subscribers are matched.

Topics are particularly appropriate for asynchronous data streams and recurring updates.

Examples include:

- Camera images.
- LiDAR measurements.
- Wheel velocity.
- Odometry.
- Robot position.
- Sensor status.
- Control-state updates.

A topic may have multiple publishers and multiple subscribers. This supports loose coupling because a publisher does not need to know the identity of every subscriber.

5. A. Topic Example

Suppose a sensor node publishes wheel-state information on /wheel_state.

Wheel State Publisher

publishes /wheel_state subscribed by

v

Speed Calculation Node

The publisher supplies the data, and the subscriber processes it.

The following commands are useful for inspecting topics:

ros2 topic list

Lists available topics.

ros2 topic info /turtle1/pose

Displays information about a topic, including its type and publisher/subscriber endpoint counts.

ros2 topic echo /turtle1/pose

Displays messages received from the topic.

These turtlesim examples require the relevant node to be running.

6. B. Advantages of Topics

Topics are useful because they:

Support asynchronous communication.

Allow multiple subscribers to receive the same information.

Decouple data producers from consumers.

Suit periodic sensor measurements and event streams.

Support distributed robotic architectures.

However, topics do not inherently provide a request-correlated response. If a client needs an explicit answer to a particular request, a service may be more appropriate.

7. V. SERVICES

A service implements request–response communication between a client and a server.

The client sends a request to a named service. The server processes that request and returns a response.

Services are appropriate for short, bounded operations, including:

- Resetting a simulator.
- Requesting a calculation.
- Changing a discrete configuration.
- Querying a component's state.
- Triggering a brief operation.

8. A. Service Communication

Service Client

|

| Request

v

Service Server

|

| Response

v

Service Client

Unlike a topic interaction, a service call has an explicit request and response relationship.

9. Inspecting Services

List available services:

ros2 service list

Inspect a service's interface type:

ros2 service type /reset

The second command is an example from turtlesim and depends on the service being available.

A service should not generally be used for a long-running operation that requires ongoing feedback or cancellation. For such operations, ROS 2 provides actions.

10. ACTIONS

An action is a communication mechanism designed for goal-oriented operations that may take time to complete.

Actions support three principal information categories:

1. Goal: The task requested by the client.
2. Feedback: Progress information provided while the task is executing.
3. Result: The outcome returned when the task finishes.

Actions also support cancellation.

11. Action Communication

Action Client

|

| Sends Goal

v

Action Server

|

| Feedback

v

Action Client

|

| Final Result

v

Action Client

For example, a navigation system may receive a goal to move a robot to a destination. During execution, the action server can provide feedback about progress. The client may cancel the goal if the destination changes.

12. B. Applications of Actions

Actions are suitable for:

- Robot navigation.
- Trajectory execution.
- Robotic arm movement.
- Long-running manipulation tasks.
- Operations requiring progress monitoring.

To list available actions:

8. ros2 action list

Actions should be selected when the task requires a goal lifecycle, feedback, or cancellation. A short operation that needs only one response is usually simpler to implement as a service.

13. VII. PARAMETERS

Parameters are named configuration values associated with a ROS 2 node. They allow developers to configure node behaviour without changing source code for every adjustment.

Typical parameters include:

- Sensor sampling frequency.
- Control thresholds.
- Robot frame identifiers.
- Algorithm settings.

- Simulation properties.
- Maximum velocity limits.

Parameters may be declared, inspected, and changed at runtime according to the node's implementation and parameter-validation rules.

14. A. Inspecting Parameters

List parameters:

ros2 param list

Retrieve a parameter value:

ros2 param get /turtlesim background_r

The example assumes that the relevant turtlesim node and parameter exist.

B. Parameters Versus Topics

Parameters are intended primarily for configuration, whereas topics are designed for message exchange.

For example, a maximum permitted speed may be configured through a parameter, while the current measured speed may be published continuously on a topic.

Using parameters for high-frequency sensor streams is generally inappropriate because their purpose differs from that of topic communication.

15. DIFFERENCE BETWEEN TOPICS, SERVICES, AND ACTIONS

The choice of communication mechanism should be based on the interaction required by the application.

Characteristic	Topic	Service	Action
Communication model	Publish–subscribe	Request–response	Goal-oriented interaction
Typical duration	Continuous or event-driven	Short, bounded operation	Potentially long-running
Direct response	Not inherent	Yes	Final result
Progress feedback	Not inherent	Not intended for ongoing feedback	Supported
Cancellation	Not inherent	Not a standard goal-cancellation mechanism	Supported
Typical example	Camera stream	Reset a simulator	Navigate to a destination

16. A. When to Use a Topic

Use a topic when information must be distributed asynchronously to one or more interested consumers.

Examples include sensor readings, odometry, and current wheel speed.

17. When to Use a Service

Use a service when a client requests an operation and needs a response.

Examples include resetting a component, requesting a calculation, or querying a value through a defined service interface.

18. When to Use an Action

Use an action when a task may take time and needs feedback, cancellation, or a final result.

Examples include navigation and trajectory execution.

The mechanisms are complementary. A robotic application commonly uses all three according to the requirements of individual operations.

19. CUSTOM INTERFACES

20. A. Purpose of Custom Interfaces

ROS 2 interfaces define the structure of data exchanged between nodes. They provide an explicit, typed contract between communicating components.

The principal interface definition types are:

- .msg — message fields.
- .srv — service request and response fields.
- .action — goal, result, and feedback fields.

Custom interfaces are useful when standard ROS 2 messages do not adequately represent the application's data.

21. B. Custom Message Example

Consider a wheel-speed calculation application that must transmit angular velocity and wheel radius.

A conceptual message definition is:

9. float64 ang_vel
10. float64 radius

The message contains two floating-point values. The publisher fills these fields and publishes the message, while the subscriber receives the data and calculates the longitudinal speed.

The calculation is:

\[v = \omega r \]

where:

- \(v\) is longitudinal speed in metres per second.
- \(\omega\) is angular velocity in radians per second.
- \(r\) is wheel radius in metres.

This example demonstrates how a custom message allows a publisher and subscriber to exchange structured data.

The message field names used in the actual .msg file must match the identifiers referenced by the application code. ROS

interface naming rules should be followed when defining the fields.

22. C. Custom Service Example

A service definition separates request fields from response fields using three hyphens:

```
# Conceptual example of a service definition
float64 input_value
---
float64 output_value
```

In this example, the client sends an input value, and the server returns a calculated output value.

The request and response types form a single service interface. The actual service must be defined in a .srv file and built through the ROS interface-generation toolchain.

23. D. Interface Package Structure

A typical custom interface package may have the following structure:

```
my_interfaces/
├── CMakeLists.txt
├── package.xml
├── msg/
│   └── WheelState.msg
└── srv/
    └── CalculateValue.srv
```

An interface package commonly uses ament_cmake, rosidl_default_generators, and rosidl_default_runtime, together with the appropriate package metadata.

After the package has been built and the workspace sourced, application nodes can import the generated interface types.

24. WORKSPACE STRUCTURE AND COLCON

ROS 2 Workspace

A workspace is a directory structure containing related ROS 2 packages under development.

A typical workspace is organised as follows:

```
ros2_ws/
│
├── src/
│   ├── publisher_package/
│   ├── subscriber_package/
│   └── interface_package/
│
├── build/
├── install/
└── log/
```

The directories serve the following purposes:

- src/ contains source code and package definitions.
- build/ contains intermediate build files.
- install/ contains installed packages and environment scripts.
- log/ contains build and test logs.

The source code and package definitions belong in src/. The other directories are generated as part of the build process.

Colcon Build System

colcon is the standard build tool commonly used to build ROS 2 workspaces.

It discovers packages, uses declared dependencies to determine build order, builds the packages, and generates installation and log outputs.

A typical workflow is:

```
Source ROS 2 environment
        |
        v
Create workspace
        |
        v
Place packages in src/
        |
        v
Run colcon build
        |
        v
Source install/setup.bash
        |
        v
Run and test the nodes
```

Fig. 2. Typical ROS 2 workspace development workflow.

Create a workspace:

```
mkdir -p ~/ros2_ws/src
```

Move into the workspace:

```
cd ~/ros2_ws
```

Build the packages:

```
colcon build
```

Source the resulting workspace overlay:

```
source install/setup.bash
```

The ROS 2 underlay should be sourced before building and using the workspace. The overlay allows the current terminal to locate packages built in the workspace.

25. ROS 2 Packages

A ROS 2 package is a unit of software organisation and distribution. It includes source files, metadata, dependencies, and build configuration.

The package.xml file specifies package metadata and dependencies.

Python packages commonly use ament_python, while C++ and interface packages commonly use ament_cmake.

Organising an application into separate packages allows interfaces, reusable libraries, and executable nodes to be maintained independently.

When a build fails, the first meaningful error should be investigated. The build logs can help identify missing dependencies, incorrect package configuration, or source-code errors.

26. QUALITY OF SERVICE (QoS)

Introduction

Quality of Service (QoS) defines how messages are stored, delivered, and managed in ROS 2.

Different robotic applications have different communication requirements. For example, a high-frequency sensor stream may prioritise low latency, while a critical state update may require reliable delivery.

Publisher and subscriber QoS settings must be compatible. A topic may be visible in the graph even when its endpoints do not successfully exchange messages because their QoS settings are incompatible.

Important QoS Policies

Policy	Meaning	Application consideration
Reliability	Reliable or best-effort delivery	Reliable delivery may suit critical data; best effort may suit high-rate streams.
Durability	Volatile or transient local	Transient local can retain samples for late-joining subscribers.
History	Keep last or keep all	Determines how message history is stored.
Depth	Queue size for keep-last history	Balances burst tolerance, memory, and latency.
Deadline	Expected interval between samples	Helps identify missed publication intervals.
Lifespan	Maximum age of a sample	Can prevent stale data from being delivered.
Liveliness	Endpoint activity assertion and monitoring	Helps detect when a publisher is no longer active.

Reliability

Reliable delivery is useful when message loss is unacceptable or costly. Best-effort delivery may be appropriate for high-rate sensor data when receiving the latest sample is more important than retransmitting every lost sample.

Durability

Volatile durability does not retain samples for late-joining subscribers through the publisher's durability history. Transient-local durability can allow late-joining subscribers to receive retained samples while the publisher remains available.

. History and Depth

The history policy determines whether the system retains a bounded number of recent messages or attempts to retain all messages. The depth setting specifies the queue size when keep-last history is used.

Large queues can increase memory use and may increase the age of data being processed. Therefore, history and depth should be selected according to the application's latency and storage requirements.

QoS Compatibility

QoS policies should be selected according to the application rather than assuming that the strictest settings are always best.

When investigating missing messages, developers should check:

1. Topic names and message types.
2. Active publisher and subscriber endpoints.
3. Namespaces and remappings.
4. ROS domain configuration.
5. QoS compatibility.

Official reference:

https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html

27. XII. PRACTICAL INSPECTION AND DEBUGGING

The ROS 2 command-line interface and graphical tools help developers understand a running application.

A. Inspecting Nodes and Topics

ros2 node list

ros2 node info /node_name

ros2 topic list

ros2 topic info /topic_name

ros2 topic echo /topic_name

These commands list active nodes, inspect node interfaces, list topics, display topic information, and show published messages.

The node and topic names should be replaced with names available in the running system.

B. Inspecting Services, Actions, and Parameters

ros2 service list

ros2 action list

ros2 param list

These commands list available services, actions, and node parameters.

Visualising the Graph

rqt_graph

This command opens the graphical ROS computation graph viewer when the relevant graphical tools are installed.

The graph can help identify unexpected node names, missing topic connections, and the relationships between publishers and subscribers.

Viewing Logs

ros2 run rqt_console rqt_console

The graphical logging interface allows developers to inspect and filter log messages.

Logging is useful for diagnosing node errors, understanding execution flow, and identifying conditions that prevent an application from behaving as expected.

The graph and logging tools should be used together with command-line inspection and application-level testing. A graph showing connected nodes does not, by itself, establish that their calculations or outputs are correct.

28. DISCUSSION

ROS 2 provides a modular architecture in which nodes communicate through typed interfaces. The computation graph represents the runtime relationships among these components.

The communication mechanism must be selected according to the application's requirements. Topics support asynchronous streams and distributed updates, services provide request–response interactions, and actions support long-running tasks with feedback and cancellation.

Parameters configure node behaviour, while custom interfaces formalise application-specific data structures. Workspaces and colcon organise package development, and QoS policies influence message delivery, retention, and endpoint compatibility.

These concepts work together. For example, two nodes may be correctly implemented but fail to exchange data if their topic names, message types, or QoS settings are incompatible. Similarly, a custom message must be generated and built before application nodes can use it.

Understanding the theory is therefore only one part of ROS 2 proficiency. Developers must also inspect the computation graph, test interfaces, build packages, and diagnose communication problems in a running environment.

CONCLUSION

ROS 2 Humble provides a framework for constructing distributed robotic applications from independent nodes. The computation graph represents their communication relationships, while topics, services, actions, and parameters support distinct application requirements.

Custom interfaces enable developers to define structured data contracts. Workspaces and colcon support package-based development, while QoS policies govern important aspects of message delivery and storage.

A sound understanding of these concepts enables developers to design modular, maintainable, and testable robotic applications. Practical proficiency requires applying these principles in working ROS 2 systems and verifying the behaviour of nodes and their communication interfaces.

29. REFERENCES

[1] Open Robotics, “Beginner: CLI Tools,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html. Accessed: Oct. 11, 2026.

[2] Open Robotics, “Beginner: Client Libraries,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html. Accessed: Oct. 11, 2026.

[3] Open Robotics, “Creating Custom ROS 2 msg and srv Files,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Single-Package-Define-And-Use-Interface.html. Accessed: Oct. 11, 2026.

[4] Open Robotics, “Creating a Workspace,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html. Accessed: Oct. 11, 2026.

[5] Open Robotics, “Creating Your First ROS 2 Package,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html. Accessed: Oct. 11, 2026.

[6] Open Robotics, “About Quality of Service Settings,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html. Accessed: Oct. 11, 2026.

[7] Open Robotics, “Understanding ROS 2 Nodes,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html. Accessed: Oct. 11, 2026.

[8] Open Robotics, “Understanding ROS 2 Topics,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html. Accessed: Oct. 11, 2026.

[9] Open Robotics, “Understanding ROS 2 Services,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html. Accessed: Oct. 11, 2026.

[10] Open Robotics, “Understanding ROS 2 Actions,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html. Accessed: Oct. 11, 2026.

[11] Open Robotics, “Understanding ROS 2 Parameters,” ROS 2 Documentation, Humble. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-

Tools/Understanding-ROS2-Parameters/Understanding-ROS2-Parameters.html. Accessed: Oct. 11, 2026.